

ЗВІТ ПО КОМАНДНОМУ ПРОЄКТУ

з дисципліни «Обробка аудіо-сигналів»

Розробка системи розпізнавання видів птахів за аудіозаписами навколишнього середовища (Kaggle BirdCLEF 2026)

Виконали: студенти групи KI-51мп
Григор'єв А.
Савчук В.
Глущенко В.

Перевірів: _____

Зміст

1	Мета роботи	2
2	Постановка задачі	2
2.1	Опис змагання BirdCLEF 2026	2
2.2	Формальна постановка	2
2.3	Склад датасету	3
3	Теоретичні відомості	3
3.1	Мел-спектрограма як вхідне представлення	3
3.2	Архітектура EfficientNetV2-S	4
3.3	Функція втрат та м'яке зважування міток	4
3.4	Аугментації даних	5
4	Хід роботи	5
4.1	Завантаження та підготовка даних	5
4.2	Стратифікована крос-валідація	5
4.3	Аудіо-пайплайн	6
4.4	Архітектура моделі	7
4.5	Навчання	7
4.6	Інференс та ансамблювання	8
5	Результати	9
5.1	Метрика оцінювання	9
5.2	Результат на публічному лідерборді	9
6	Висновки	10
7	Напрями подальших досліджень	10
7.1	Боротьба з дисбалансом класів через мульти-кроп	10
7.2	Навчана голова поверх ансамблю	11
7.3	Порогова фільтрація слабких прогнозів	11
7.4	Псевдо-лейблінг тестових soundscapes	11

1. Мета роботи

Метою командного проєкту є розробка повного ML-пайплайну для автоматичного розпізнавання видів птахів за аудіозаписами пасивного акустичного моніторингу в рамках змагання Kaggle BirdCLEF 2026. У межах проєкту реалізовано: завантаження та аналіз датасету, побудову аудіо-пайплайну на основі бібліотеки torchaudio, навчання моделі EfficientNetV2-S з трансферним навчанням, стратифіковану крос-валідацію та формування фінального ансамблевого прогнозу для сабмішну.

2. Постановка задачі

2.1. Опис змагання BirdCLEF 2026

BirdCLEF — щорічне змагання на платформі Kaggle, організоване Cornell Lab of Ornithology спільно з Google. Основна мета — автоматична ідентифікація видів птахів із записів пасивного акустичного моніторингу (soundscapes). Складність задачі зумовлена: значним дисбалансом класів (окремі види мають лічені тренувальні приклади), накладанням декількох видів в одному записі, шумами вітру та техногенного походження, а також необхідністю обробки неперервних звукових доріжок ковзним вікном.

2.2. Формальна постановка

Нехай $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N$ — навчальна вибірка, де $\mathbf{x}_i \in \mathbb{R}^T$ — моноаудіосигнал тривалістю T семплів, а $\mathbf{y}_i \in \{0, 1\}^C$ — багатомітковий вектор присутності видів ($C = 234$ класи). Задача — навчити класифікатор $f_\theta : \mathbb{R}^T \rightarrow [0, 1]^C$, що мінімізує бінарну крос-ентропію:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C \left[y_{ic} \log \sigma(z_{ic}) + (1 - y_{ic}) \log(1 - \sigma(z_{ic})) \right],$$

де $z_{ic} = f_\theta(\mathbf{x}_i)_c$ — логіт, σ — сигмоїда. Метрика змагання — макросереднений ROC-AUC лише по класах, присутніх у вибірці оцінки:

$$\text{AUC}_{\text{macro}} = \frac{1}{|\mathcal{C}^+|} \sum_{c \in \mathcal{C}^+} \text{AUC}_c, \quad \mathcal{C}^+ = \{c \mid \sum_i y_{ic} > 0\}.$$

2.3. Склад датасету

Табл. 1. Основні характеристики датасету BirdCLEF 2026

Параметр	Значення
Кількість класів (видів)	234
Основні тренувальні кліпи	індивідуальні записи птахів (.ogg)
Soundscapes для навчання	неперервні записи з часовими мітками (train_soundscapes)
Тестові soundscapes	неперервні записи без міток (test_soundscapes)
Цільова частота дискретизації	32 000 Гц
Тривалість аналізованого вікна	5 секунд

3. Теоретичні відомості

3.1. Мел-спектрограма як вхідне представлення

Для подачі аудіосигналу до згорткової нейронної мережі його перетворюють у двовимірне часово-частотне зображення. Стандартна процедура включає чотири кроки.

Спочатку виконується коротке перетворення Фур'є (STFT) з вікном довжини N_{FFT} та кроком H :

$$X[k, t] = \sum_{n=0}^{N_{\text{FFT}}-1} x[tH + n] w[n] e^{-j2\pi kn/N_{\text{FFT}}}.$$

Потім застосовується банк мел-фільтрів $\{H_m\}$ (нормалізація Slaney):

$$S_m[t] = \sum_k H_m[k] \cdot |X[k, t]|^2.$$

Результат переводиться у логарифмічний масштаб:

$$\tilde{S}_m[t] = 10 \cdot \log_{10}(S_m[t] + \varepsilon).$$

Нарешті виконується поелементна нормалізація до $[0, 1]$:

$$S^*[m, t] = \frac{\tilde{S}[m, t] - \min \tilde{S}}{\max \tilde{S} - \min \tilde{S} + 10^{-6}}.$$

Параметри проєкту: $SR = 32\,000$ Гц, $N_{FFT} = 1024$, $H = 320$, $M = 128$ мел-смуг, $f_{min} = 50$ Гц, $f_{max} = 14\,000$ Гц.

3.2. Архітектура EfficientNetV2-S

EfficientNetV2 [?] — вдосконалена версія EfficientNet, що поєднує операції Fused-MBConv на ранніх стадіях та MBConv на пізніх. Версія S налічує близько 21 млн параметрів. Ключові особливості архітектури:

- **Fused-MBConv**: заміна пари (depthwise + pointwise) на одну звичайну згортку для перших стадій — прискорює навчання без втрати якості.
- **Squeeze-and-Excitation**: механізм каналної уваги, що адаптивно зважує карти ознак.
- **Одноканальний вхід**: $in_chans = 1$, тобто мел-спектрограма подається як одноканальне зображення 224×224 .

Модель ініціалізована чекпоінтом, попередньо навченим на ImageNet-21k та дотренованим на великому корпусі даних BirdCLEF попередніх років (tf_efficien

3.3. Функція втрат та м'яке зважування міток

Оскільки кожен аудіофрагмент може містити кілька видів, застосовується BCE з логітами. Для врахування вторинних міток використовується схема м'якого зважування:

Табл. 2. Схема зважування міток

Тип мітки	Джерело	Вага w
Основна мітка кліпу	primary_label	1.0
Вторинна мітка кліпу	secondary_labels	0.3
Мітка soundscape-сегменту	train_soundscape_labels.csv	1.0

3.4. Аугментації даних

MixUp ($\alpha = 0.4$): лінійна інтерполяція пар зразків у просторі зображень та міток:

$$\tilde{\mathbf{x}} = \lambda \mathbf{x}_i + (1 - \lambda) \mathbf{x}_j, \quad \tilde{\mathbf{y}} = \lambda \mathbf{y}_i + (1 - \lambda) \mathbf{y}_j, \quad \lambda \sim \text{Beta}(\alpha, \alpha).$$

SpecAugment [?]: маскування прямокутних областей мел-спектрограми нулями — 2 маски по частотній осі (до 30 смуг кожна) та 2 маски по часовій осі (до 40 кроків кожна).

4. Хід роботи

4.1. Завантаження та підготовка даних

Після зчитування `train.csv` та `taxonomy.csv` виконується:

1. Побудова словника `label2idx` для 234 класів на основі стовпців `sample_subm`
2. Парсинг вторинних міток кожного запису (рядок у форматі `['bird1', 'bird2']`).
3. Обробка `soundscape`-анотацій: для кожного рядка міток визначаються часові межі вікна `[start_sec, end_sec]`.
4. Виявлення класів з кількістю записів < 14 — вони потрапляють лише у тренувальну множину і не включаються у валідацію.

4.2. Стратифікована крос-валідація

Для коректного 5-фолдового розбиття реалізовано два алгоритми.

Балансований однолейбловий розподіл (`assign_balanced_folds_single`): для кожного класу записи перемішуються та рівномірно розподіляються по фолдах за допомогою залишку від ділення $i \bmod K$.

Жадібний багателейбловий розподіл (`assign_multilabel_folds`): для `soundscape`-записів кожен зразок призначається до фолду, що мінімізує зважений `score` відхилення:

$$\text{score}(\text{fold}) = \Delta_{\text{label}} + 0.2 \cdot \Delta_{\text{size}},$$

де рідкісні мітки (з малою частотою у датасеті) мають вищий пріоритет при сортуванні.

4.3. Аудіо-пайплайн

Реалізовано дві функції завантаження: `load_and_crop` для тренувальних кліпів (випадковий кроп під час навчання, центральний — на валідації) та `load_exact_window` для `soundscape`-сегментів з точними часовими межами.

Listing 1. Завантаження та кропінг кліпу

```
1 def load_and_crop(filepath, sr=SR, duration=DURATION,
2                     is_train=True) -> np.ndarray:
3     target_len = int(sr * duration)
4     wav, file_sr = torchaudio.load(str(filepath))
5     wav = wav.mean(dim=0) # стерео -> моно
6     if file_sr != sr:
7         wav = F.resample(wav, file_sr, sr) # ресемплінг
8     y = wav.numpy()
9     if len(y) < target_len:
10         y = np.tile(y, math.ceil(target_len / len(y)))
11     start = random.randint(0, len(y) - target_len) \
12         if is_train else (len(y) - target_len) // 2
13     return y[start : start + target_len].astype(np.float32)
```

Listing 2. Перетворення у мел-спектрограму

```
1 mel_transform = T.MelSpectrogram(
2     sample_rate=SR, n_fft=N_FFT, hop_length=HOP_LENGTH,
3     n_mels=N_MELS, f_min=FMIN, f_max=FMAX, power=2.0,
4     norm='slaney', mel_scale='slaney',
5 ).to(device)
6
7 def audio_to_melspec(y: np.ndarray) -> np.ndarray:
8     wav = torch.from_numpy(y).unsqueeze(0).to(device)
9     mel = mel_transform(wav)
10    mel_db = T.AmplitudeToDB(stype='power')(mel).squeeze(0)
11    # нормалізація до [0, 1] по кожному зразку
12    mel_db = (mel_db - mel_db.min()) / \
13        (mel_db.max() - mel_db.min() + 1e-6)
14    return mel_db.cpu().numpy().astype(np.float32)
```

Результуюча мел-спектрограма ($128 \times T'$) масштабується до 224×224 через білінійну інтерполяцію та подається у модель як одноканальне зображення.

4.4. Архітектура моделі

Listing 3. Клас BirdModel

```
1 class BirdModel(nn.Module):
2     def __init__(self):
3         super().__init__()
4         self.backbone = timm.create_model(
5             'tf_efficientnetv2_s.in21k',
6             pretrained=False,
7             in_chans=1,          # одноканальний вхід
8             num_classes=N_CLASSES,
9         )
10        # завантаження власного чекпоїнту
11        ckpt = torch.load(MODEL_PATH, map_location='cpu')
12        state_dict = fix_keys(ckpt.get('state_dict', ckpt))
13        self.backbone.load_state_dict(state_dict, strict=False)
14
15    def forward(self, x):
16        return self.backbone(x)
```

Функція `fix_keys` усуває невідповідності назв шарів між форматом чекпоїнту та реалізацією `timm`: видаляє префікс `model.`, нормалізує назви стовбурового шару та стадій.

4.5. Навчання

Гіперпараметри навчання наведено в таблиці 3.

Табл. 3. Гіперпараметри навчання

Параметр	Значення
Максимальна кількість епох	10
Розмір батчу	32
Швидкість навчання	5×10^{-4}
Weight decay	10^{-4}
Gradient clipping (norm)	1.0
Кількість фолдів	5
Early stopping (patience)	1
Змішана точність (AMP)	увімкнено
MixUp (α)	0.4
SpecAugment	2 freq + 2 time маски

Навчання ведеться з `WeightedRandomSampler`: кожному зразку призначається вага, обернена до середньої частоти його міток, що компенсує дисбаланс класів. Застосовується AMP (`autocast` + `GradScaler`) для прискорення на GPU. Чекпоїнт зберігається лише при покращенні валідаційного AUC.

Listing 4. Основний цикл тренувального кроку

```

1 if USE_MIXUP:
2     images, targets = mixup_data(images, targets)
3     optimizer.zero_grad()
4     with autocast():
5         loss = criterion(model(images), targets)
6     scaler.scale(loss).backward()
7     scaler.unscale_(optimizer)
8     torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
9     scaler.step(optimizer)
10    scaler.update()
11 if scheduler:
12    scheduler.step()

```

4.6. Інференс та ансамблювання

Тестові `soundscapes`-файли обробляються ковзним вікном за розкладом `sample_submission`. Для кожного 5-секундного вікна будується мел-спектрограма (батч-обробка на GPU), після чого виконується `forward pass`. Фінальний прогноз — рівноважне

усереднення ймовірностей по 5 фолд-моделях:

$$\hat{p}_c = \frac{1}{K} \sum_{k=1}^K \sigma(z_c^{(k)}), \quad K = 5.$$

Listing 5. Ансамблеве усереднення (submit ноутбук)

```
1 for fold_idx, (weights_path, fold_w) in enumerate(  
2     zip(FOLD_WEIGHTS, FOLD_ENSEMBLE_WEIGHTS)):  
3     model.load_state_dict(torch.load(weights_path,  
4                                     map_location=device))  
5     model.eval()  
6     # ... inference loop ...  
7     ensemble_probs += fold_probs * fold_w  
9 all_probs = ensemble_probs / sum(FOLD_ENSEMBLE_WEIGHTS)
```

Перед збереженням `submission.csv` виконуються автоматичні перевірки: відповідність кількості рядків та стовпців зразку, відсутність NaN, коректний діапазон ймовірностей $[0, 1]$.

5. Результати

5.1. Метрика оцінювання

Офіційною метрикою змагання BirdCLEF 2026 є макро-усереднений ROC-AUC по класах, присутніх у прихованій тестовій вибірці. Платформа Kaggle повертає **публічний скор** (public leaderboard), що розраховується на підмножині тестових soundscapes, відкритій для оцінки під час змагання.

5.2. Результат на публічному лідерборді

Фінальний сабмішн сформовано як рівноважний ансамбль 5 фолд-моделей EfficientNet-S (ноутбук `birdclef submit cross -- effnet ensemble`, автор — Andrii Grygoriev). Результат наведено в таблиці 4.

Табл. 4. Результат фінального сабмішну на публічному лідерборді

Модель / конфігурація	Public AUC
EfficientNetV2-S, 5-fold ensemble	0.897

Досягнутий показник ROC-AUC = 0.897 свідчить про високу якість розробленого пайплайну: модель впевнено розрізняє присутність та відсутність видів у більшості класів тестової вибірки.

6. Висновки

У ході командного проєкту розроблено повний ML-пайплайн для задачі класифікації птахів за аудіозаписами. Отримано такі результати:

1. Реалізовано ефективний аудіо-пайплайн на основі torchaudio: ресемплінг до 32 кГц, побудова мел-спектрограм з параметрами, оптимізованими для аудіо птахів ($f_{\min} = 50$ Гц, $f_{\max} = 14$ кГц, 128 мел-смуг), нормалізація, масштабування до 224×224 .
2. Побудовано систему аугментацій — MixUp ($\alpha = 0.4$) та SpecAugment (4 маски) — що зменшує перенавчання для рідких видів із малою кількістю тренувальних зразків.
3. Розроблено стратегію м'якого зважування міток (вторинні — вага 0.3, soundscape — 1.0), що дозволяє ефективно задіяти додаткову анотацію.
4. Реалізовано жадібну многолейблову стратифікацію для коректного 5-фолдового розбиття soundscape-даних з урахуванням рідкості класів.
5. Навчено ансамбль із 5 фолд-моделей EfficientNetV2-S з трансферним навчанням від великого корпусу BirdCLEF; сформовано та верифіковано фінальний сабмішн.

Результати підтверджують ефективність підходу «аудіо → мел-спектрограма → CNN» для задач акустичного моніторингу біорізноманіття.

7. Напрями подальших досліджень

Незважаючи на досягнуті результати, виявлено кілька перспективних напрямів для покращення якості системи.

7.1. Боротьба з дисбалансом класів через мульти-кроп

Поточна реалізація load_and_crop обирає один випадковий 5-секундний відрізок з кожного запису за епоху. Для андерсемплованих класів це означає, що

за умов `WeightedRandomSampler` той самий зразок може з'являтися багаторазово, але завжди з однаковим вмістом. Пропонується доповнити семплер логікою мульти-кропінгу: для класів з кількістю записів нижче певного порогу $n_{\min}$ один запис може бути представлений кількома різними часовими вікнами в межах однієї епохи. Це збільшить різноманіття ефективних зразків для рідкісних видів без штучного дублювання і покращить покриття звукового матеріалу без зміни розміру датасету.

7.2. Навчана голова поверх ансамблю

Поточне ансамблювання здійснюється як просте (або зважене) усереднення ймовірностей фолд-моделей. Більш потужним підходом є заміна усереднення на навчану агрегацію: конкатенація виходів K фолдів утворює вектор розмірності $K \times C$, поверх якого тренується невелика мережа (наприклад, 1–2 повнозв'язних шари з активацією ReLU та dropout). Така «stacking-голова» здатна навчитися, яким фолдам довіряти більше в залежності від класу, та скоригувати систематичні зміщення окремих моделей.

7.3. Порогова фільтрація слабких прогнозів

Аналіз розподілу вихідних ймовірностей показує наявність великої кількості ненульових, але дуже малих значень (порядку 10^{-3} і менше). З огляду на те, що метрика змагання є AUC (а не ассигасу), такі значення безпосередньо не шкодять, проте їх обнулення за порогом τ (наприклад, $\tau = 0.01$) може підвищити розбірливість прогнозів і потенційно покращити ранжування. Пропонується провести пошук оптимального τ на валідаційній вибірці:

$$\hat{p}_c^\tau = \hat{p}_c \cdot \mathbf{1}[\hat{p}_c \geq \tau].$$

7.4. Псевдо-лейблінг тестових soundscapes

Псевдо-лейблінг є напівнаглядним методом, що дозволяє залучити нерозмічені тестові дані до навчання. На першому кроці модель генерує «м'які» прогнози на тестових soundscapes; зразки з високою впевненістю ($\max_c \hat{p}_c \geq \tau_{\text{pl}}$) додаються до тренувальної множини з відповідними псевдо-мітками. Потім модель донавчається на розширеному датасеті. Цикл може повторюватись ітеративно. Метод є ефективним, проте вимагає ретельного підбору порогу τ_{pl} та механізмів фільтрації шумних псевдо-міток, що збільшує складність пайплайну.